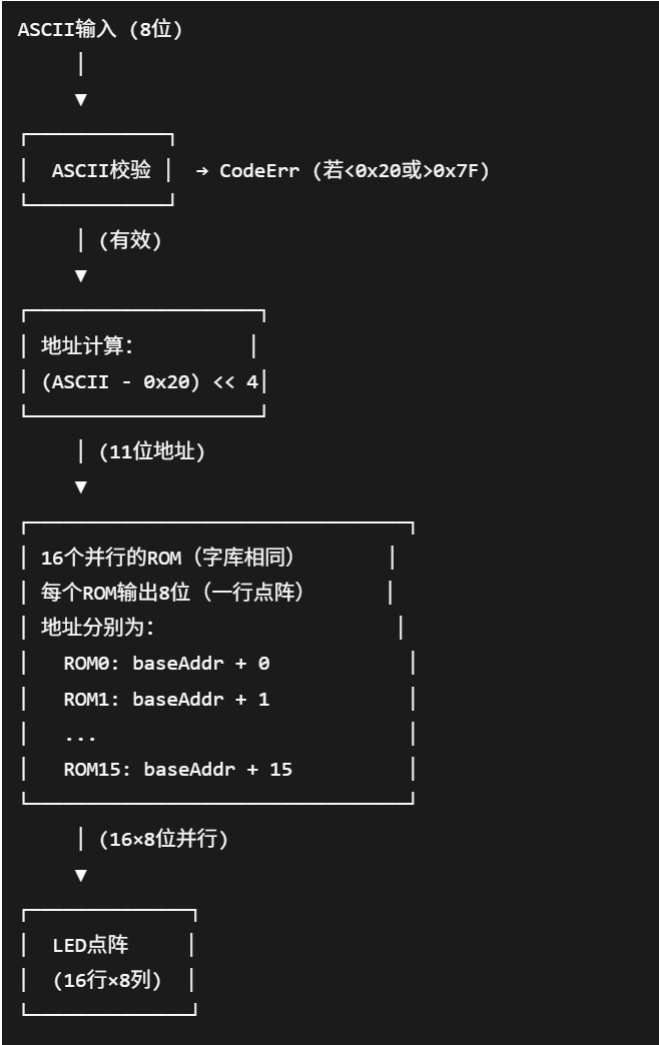


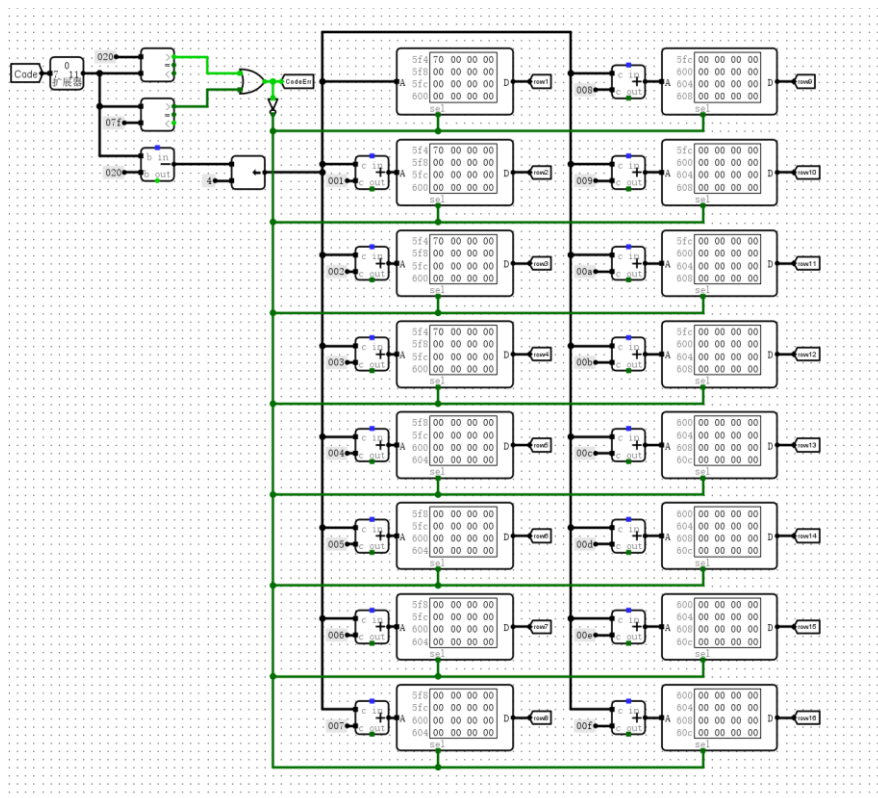
实验一 只读存储器：

1.原理图：



设计思路同原理图。

2.电路图：



3.子模块功能描述

ROM

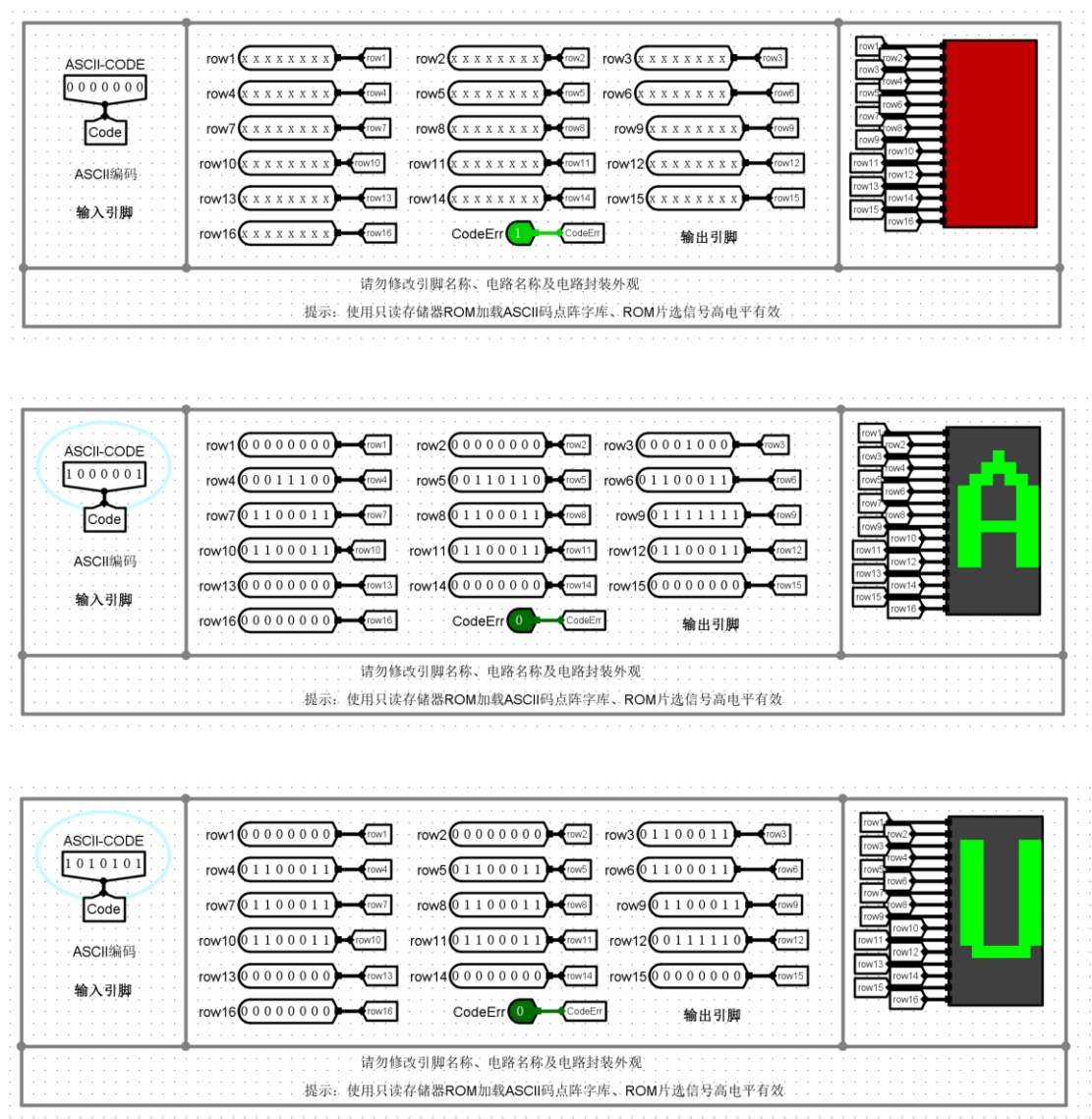
方向	信号名	位宽	功能说明
输入	Addr	11 bit	存储器地址输入
输入	Sel	1 bit	片选信号，高电平有效
输出	Dout	8 bit	点阵数据输出

4. 电路功能：

检测输入的 ASCII 是否为可见字符（是否在 0x20-0x7F 之间），如果是则将对应

字符用 LED 阵列显示。

5. 仿真模拟：

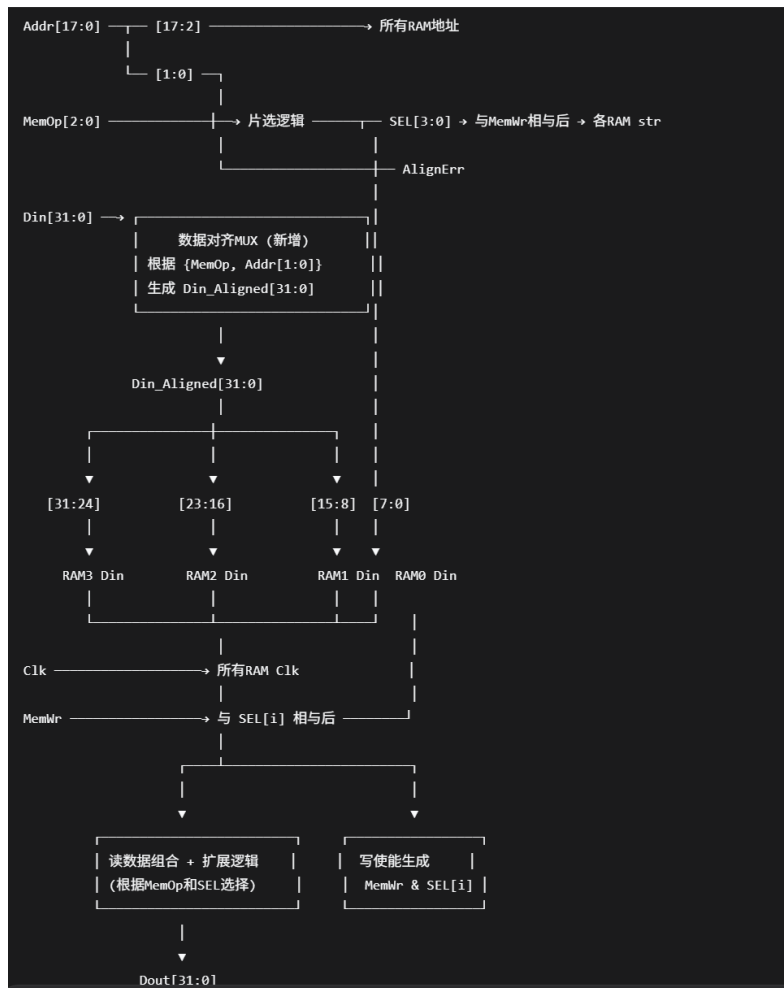


6. 错误现象及分析：

在完成本次实验的过程中，没有遇到任何错误。

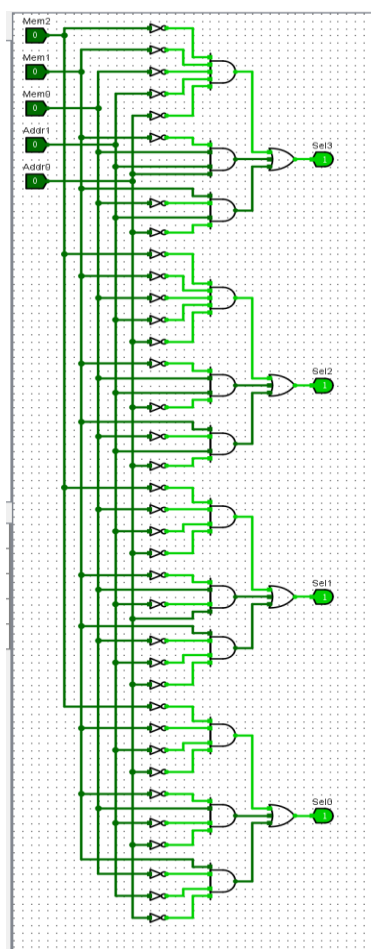
实验二 数据存储器实验

1.原理图：



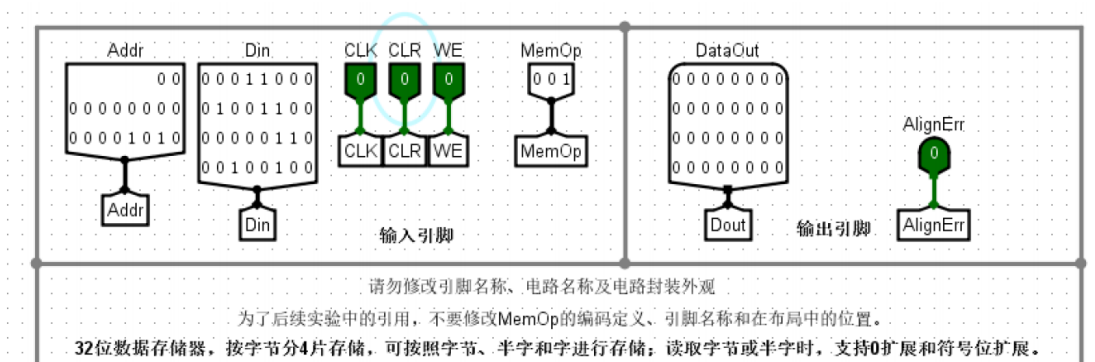
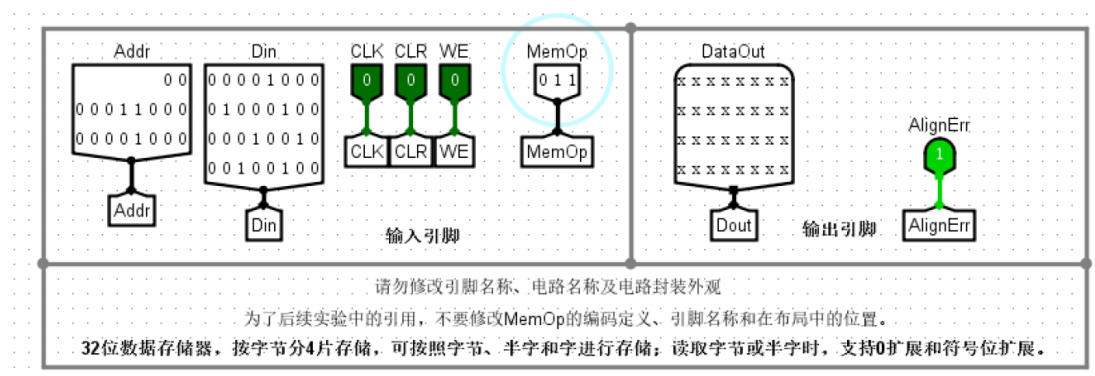
2.电路图：

SEL



SEL 模块接口功能表

方向	信号名	位宽	功能说明
输入	MemOp	3 bit	访存类型控制信号
输入	Addr[1:0]	2 bit	地址最低两位
输出	SEL3	1 bit	第 3 字节 RAM 片选（最高字节）
输出	SEL2	1 bit	第 2 字节 RAM 片选

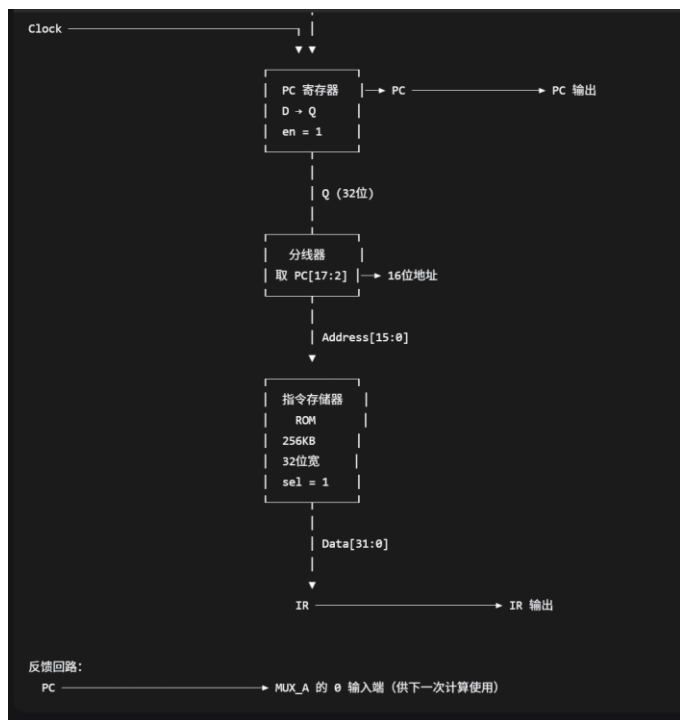
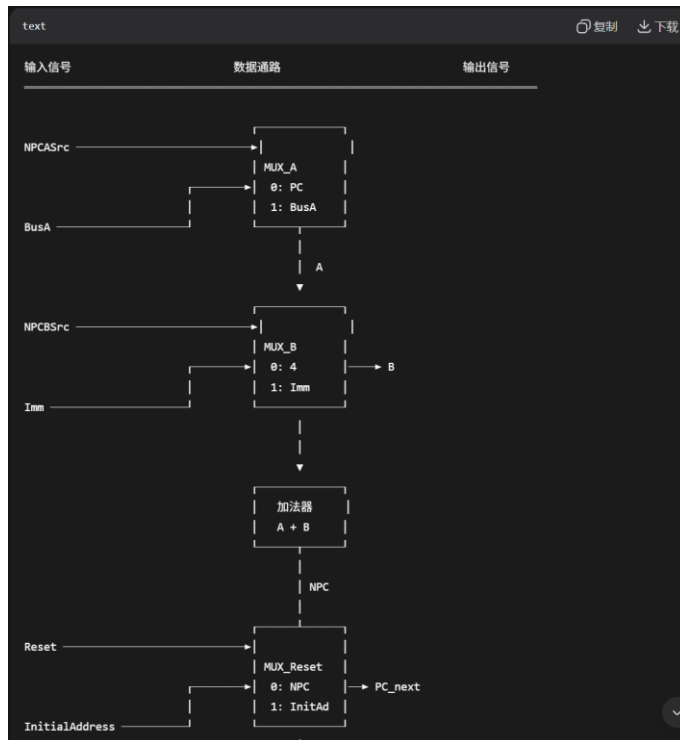


5. 错误现象及分析：

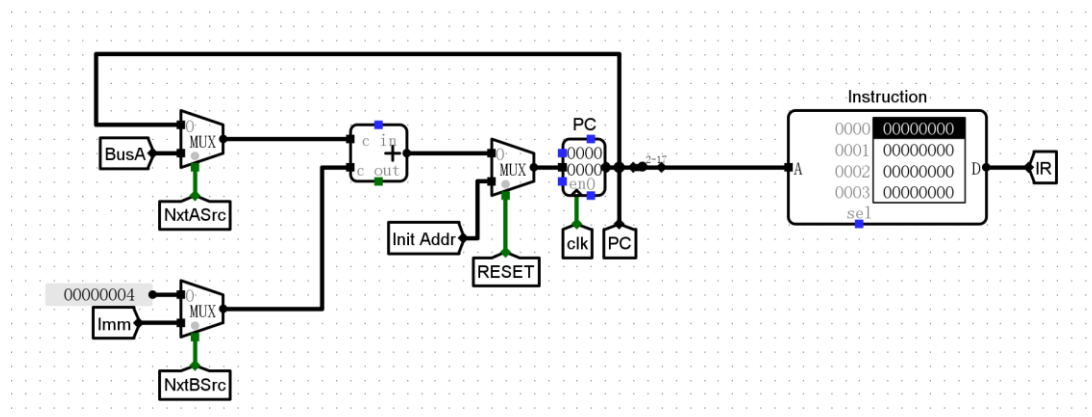
一开始没能正确理解 Addr 与 MemOp 的含义，导致设计出现失误。后来查阅资料后完成了正确的设计。

实验三 取指令部件 IFU 实验

1. 原理图：



2. 电路图:



3. 电路功能：

根据初始地址 Initial Address、立即数寄存器 Imm 和 rs1 寄存器的数据 BusA，以及控制信号 Reset、NxtASrc 和 NxtBSrc 的赋值输出当前指令的地址寄存器 PC 和指令存储器的输出内容 IR。

在程序执行过程中，下一条指令地址的计算有多种情形：

- 1) 顺序执行指令，则 $PC = PC + 4$ 。
- 2) 无条件跳转指令，jal 指令， $PC = PC + \text{立即数 imm}$ ；jalr 指令， $PC = R[\text{rs1}] + \text{立即数 imm}$ 。
- 3) 分支转移指令，根据比较运算的结果和 Zero 标志位来判断，如果条件成立则 $PC = PC + \text{立即数 imm}$ ，否则 $PC = PC + 4$

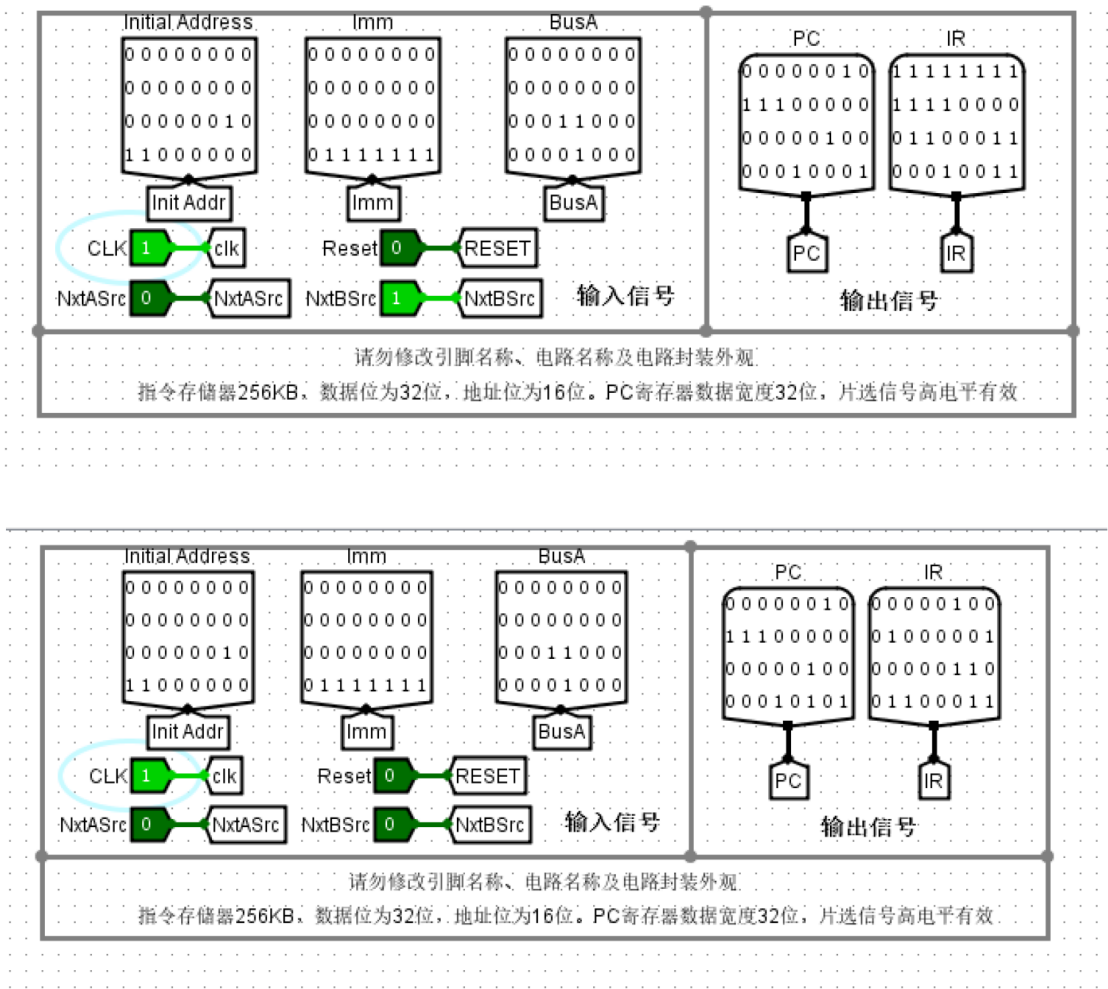
NxtASrc 和 NxtBSrc 赋值定义表

表 5.3 NxtASrc 和 NxtBSrc 赋值定义表

指令类型	NxtASrc	NxtBSrc	下条指令地址
顺序指令	0	0	$PC = PC + 4$
无条件跳转 jal	0	1	$PC = PC + \text{Imm}$
无条件跳转 jalr	1	1	$PC = R[\text{rs1}] + \text{Imm}$
分支跳转指令	0	1	条件成立 $PC = PC + \text{Imm}$ ，否则 $PC = PC + 4$

当 NxtASrc=0 时，选择 PC 寄存器的值，否则选择 Rs1 寄存器值 BusA。当 NxtBSrc =0 时选择常量 4，否则选择立即数 Imm。

4. 仿真测试：



5. 错误现象及分析：

在完成时间的过程中，没有遇到任何错误。

6. 测试表格：

序号	Reset	Initial Address	Imm	BusA	NPCASrc	NPCBSrc	PC	IR
1	1	0x400	0	0	0	0	0	0

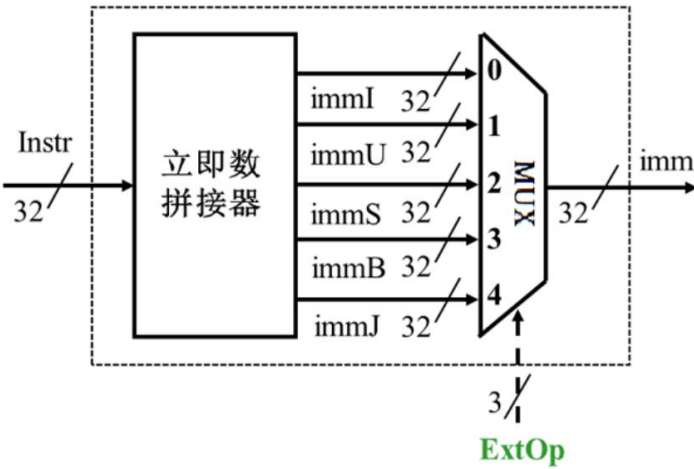
[illegible]

序号	Reset	Initial Address	Imm	BusA	NPCASrc	NPCBSrc	PC	IR
16	0	0	0xffff9c8	0	0	1	0x0c08	0xffc10113
17							0x5d0	0xdeadbeef

实验四 取操作数部件 IDU 实验

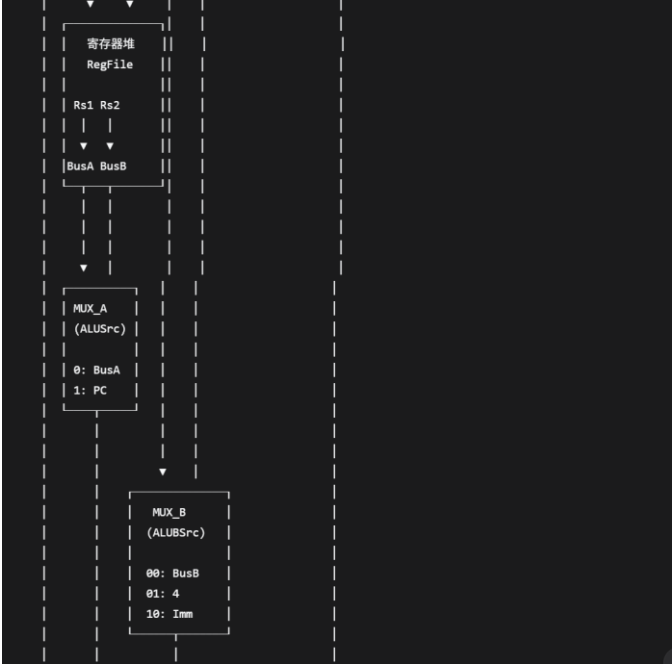
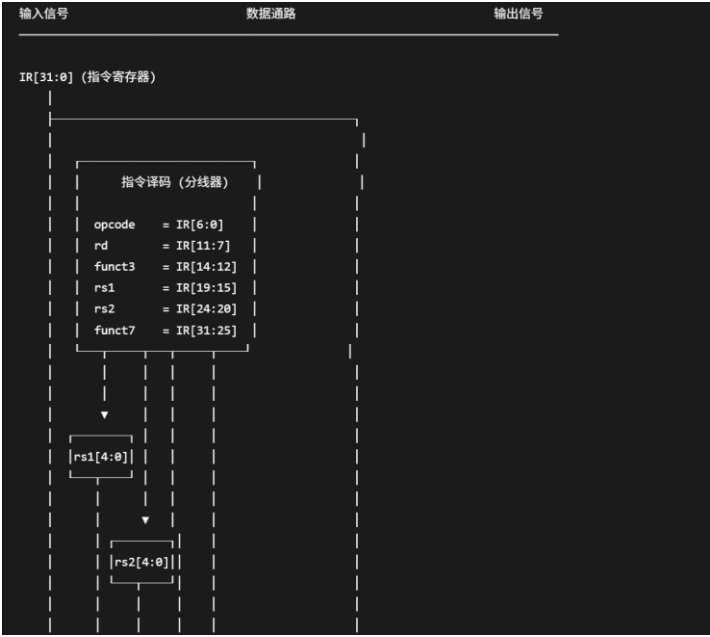
1. 原理图：

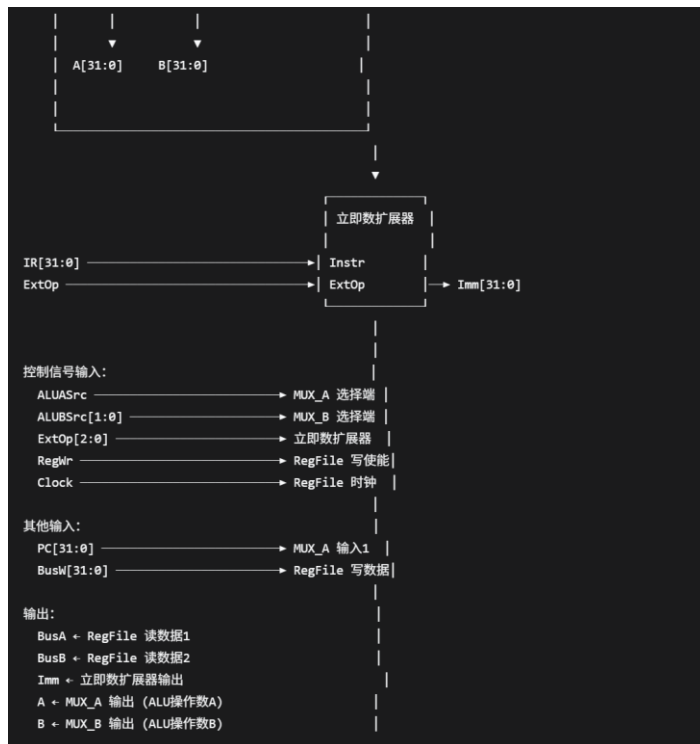
立即数扩展器：



寄存器堆：原理图大致同 lab3.4 将 0 号寄存器强制设为 0

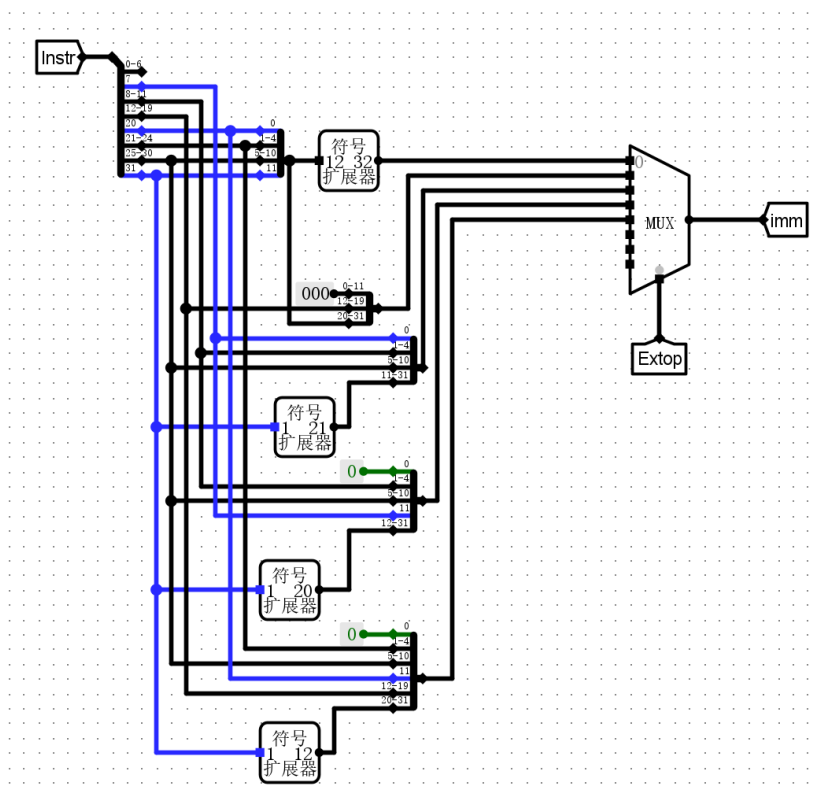
IDU：



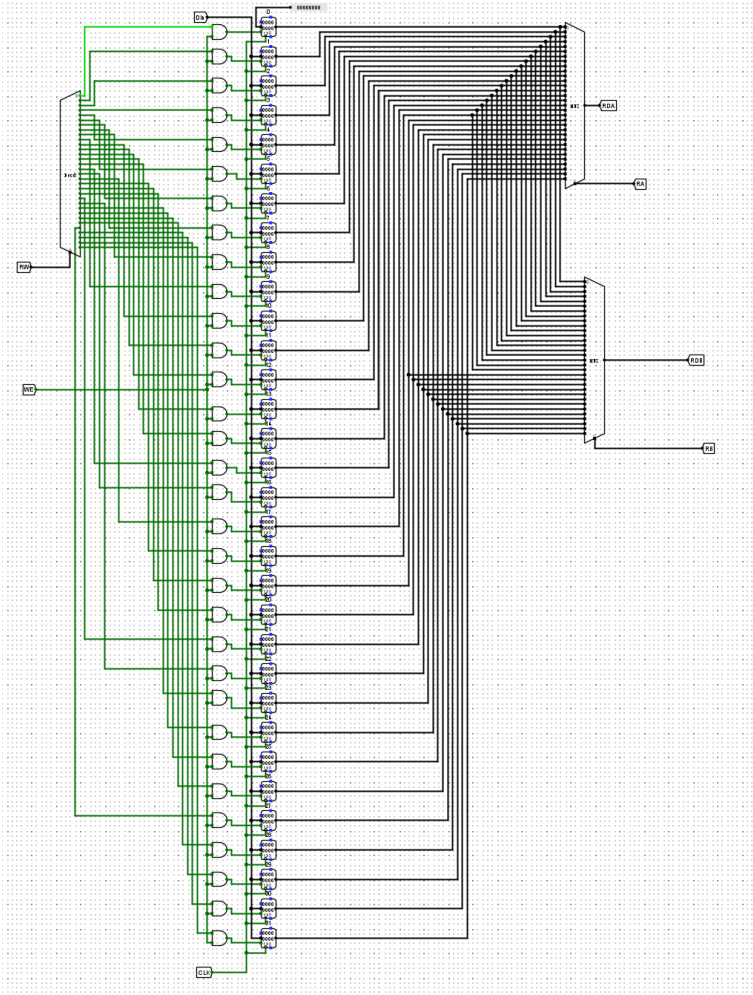


2. 电路图:

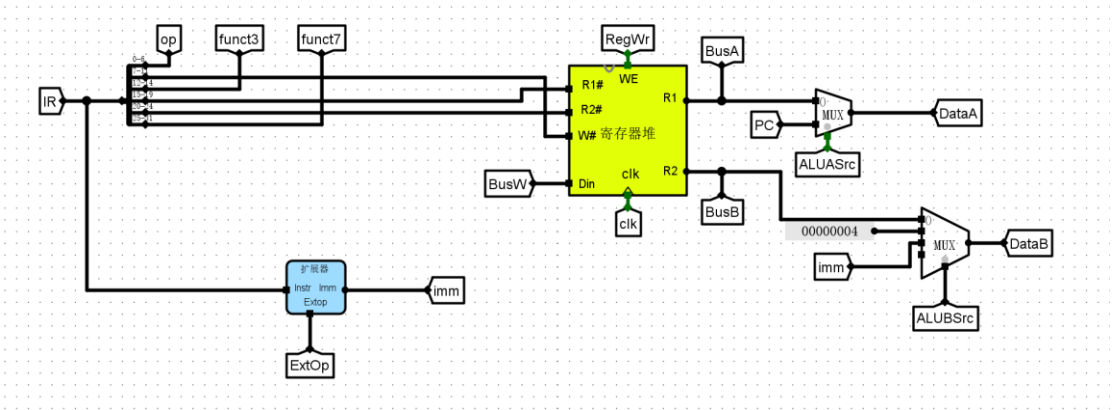
立即数扩展器:



寄存器堆：



IDU:



3. 电路功能：

ALUASrc 和 ALUBsrc 用来控制两个多路选择器的输出数据，ALUASrc 控

制 1 个两路选择器，当 ALUASrc=0 时，选择 BusA 输出到 ALU 的操作数 A 口，当 ALUASrc=1 时输出 PC。ALUBSrc 控制 1 个四路选择器，当 ALUBSrc=00 时选择 BusB 输出到 ALU 的操作数 B 口，当 ALUBSrc=01 时选择输出常数 4，当 ALUBSrc=10 时选择输出 32 位立即数。

通过控制信号 ExtOp 来选择不同立即数编码类型以及在扩展器中进行的扩展操作。

4. 仿真测试：

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
0x00	fedca2b7	0xfedca000	ExtOp=1, RegWr=1, ALUASrc=0, ALUBSrc=2	lui x5, 0xfedca	0x00000000	0xfedca000
0x04	f9c28293	0xfedc9f9c	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=2	addi x5, x5, -100	0xfedca000	0xffffffff9c
0x08	01006013	0x00000010	ExtOp=0, RegWr=1,	ori x0,	0x00000000	0x00000010

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
			ALUASrc=0, ALUBSrc=2	x0, 16		
0x0c	06502223	0x00000064	ExtOp=2, RegWr=0, ALUASrc=0, ALUBSrc=2	sw x5, 100(x0)	0x00000000	0x00000064
0x10	06400303	0xfffff9c	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=2	lb x6, 100(x0)	0x00000000	0x00000064
0x14	06601423	0x00000068	ExtOp=2, RegWr=0, ALUASrc=0, ALUBSrc=2	sh x6, 104(x0)	0x00000000	0x00000068
0x18	06405383	0x00009f9c	ExtOp=0, RegWr=1, ALUASrc=0,	lhu x7, 100(x0)	0x00000000	0x00000064

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
			ALUBSrc=2			
0x1c	06701623	0x0000006c	ExtOp=2, RegWr=0, ALUASrc=0, ALUBSrc=2	sh x7, 108(x0)	0x00000000	0x0000006c
0x20	4042d413	0xffedc9f9	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=2	srai x8, x5, 4	0xffedc9f9c	0x00000404
0x24	006444b3	0x00123665	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=0	xor x9, x8, x6	0xffedc9f9	0xfffff9c
0x28	00649533	0x50000000	ExtOp=0, RegWr=1, ALUASrc=0,	sll x10, x9, x6	0x00123665	0xfffff9c

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
			ALUBSrc=0			
0x2c	008505b3	0x4fedc9f9	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=0	add x11, x10, x8	0x50000000	0xffedc9f9
0x30	00b2a633	0x00000001	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=0	slt x12, x5, x11	0xfedc9f9c	0x4fedc9f9
0x34	00b2b6b3	0x00000000	ExtOp=0, RegWr=1, ALUASrc=0, ALUBSrc=0	sltu x13, x5, x11	0xfedc9f9c	0x4fedc9f9
0x38	40b287b3	0xaaeed5a3	ExtOp=0, RegWr=1, ALUASrc=0,	sub x15, x5, x11	0xfedc9f9c	0x4fedc9f9

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
			ALUSrc=0			
0x3c	06f02823	0x00000070	ExtOp=2, RegWr=0, ALUASrc=0, ALUSrc=2	sw x15, 112(x0)	0x00000000	0x00000070
0x40	0067c263	0x00000001	ExtOp=3, RegWr=0, ALUASrc=0, ALUSrc=0	blt x15, x6, 4	0xaeed5a3	0xfffff9c
0x44	0067d263	0x00000001	ExtOp=3, RegWr=0, ALUASrc=0, ALUSrc=0	bge x15, x6, 4	0xaeed5a3	0xfffff9c
0x48	0040086f	0x0000004c	ExtOp=4, RegWr=1, ALUASrc=1,	jal x16, 16	0x00000048	0x00000004

PC	IR	BusW	控制信号	指令功能汇编语句	DataA	DataB
			ALUSrc=1			
0x4c	004808e7	0x00000050	ExtOp=0, RegWr=1, ALUASrc=1, ALUSrc=1	jalr x17, x16, 4	0x0000004c	0x00000004
0x50	00001917	0x00001050	ExtOp=1, RegWr=1, ALUASrc=1, ALUSrc=2	auipc x18, 1	0x00000050	0x00001000

5. 错误现象及分析：

在完成实验的过程中，没有遇到任何错误。

实验五 数据通路实验

1. 原理图：

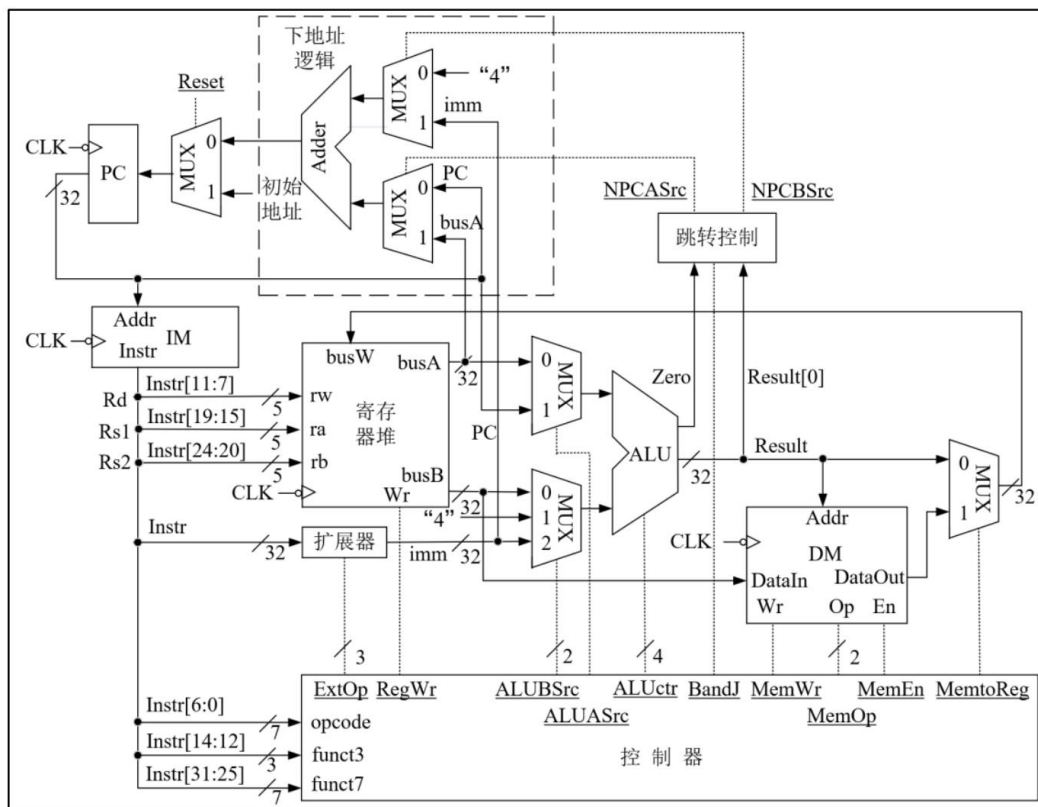
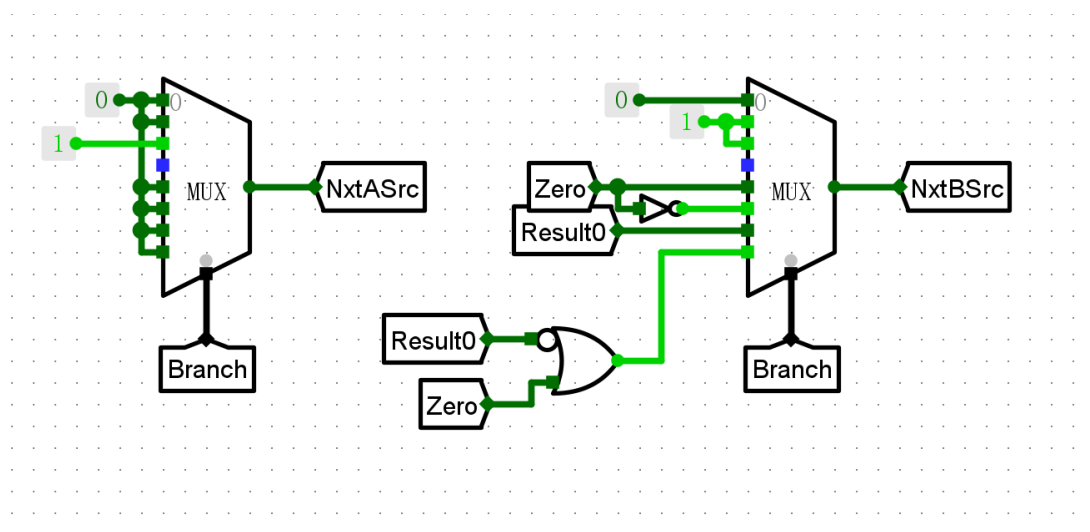


图 5.7 单周期 CPU 设计原理图

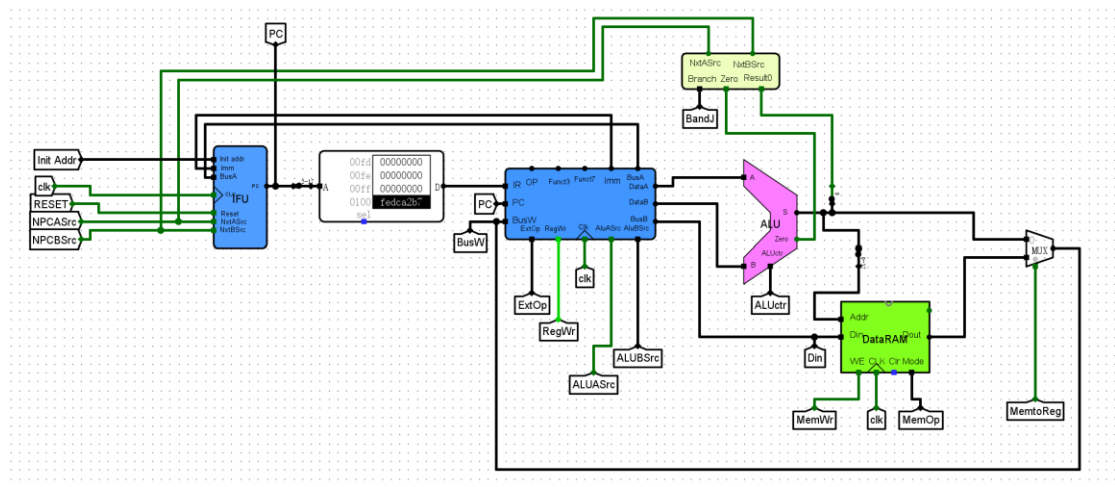
2. 电路图:

本实验大量添加了之前已经完成的子电路，在此只列举新增电路的电路图。

BandJ:



Main:



3. 电路功能：

本实验数据通路是 RV32I 单周期 CPU 完整执行通路，核心功能是按指令流程完成取指、译码、取数、运算、访存、写回全过程。

1. **取指令 (IF)**：由 PC 给出地址，从指令存储器 ROM 读出指令，下地址逻辑根据跳转 / 分支信号计算下一 PC 值。
2. **指令译码与取数 (ID)**：分解指令字段，通过立即数扩展器生成 32 位立即数，从寄存器堆读出 BusA、BusB 操作数。
3. **执行 (EX)**：ALU 完成算术、逻辑、比较运算，跳转控制器根据指令与标志位输出 NPCASrc、NPCBsrc 控制信号。
4. **访存 (MEM)**：数据存储器 RAM 支持按字节 / 半字 / 字读写，根据 MemOp 完成符号 / 零扩展与字节选择。
5. **写回 (WB)**：将 ALU 运算结果或存储器读出数据，写回寄存器堆对应目的寄存器。

整体实现 RV32I 运算、访存、分支、跳转全指令的数据流传输与控制，单周期完成一条指令执行。

4. 仿真测试：

序号	指令	控制信号赋值，未列出的控制信号值为 0。	输出信号值 PC、BusW、Din、NPCASrc、NPCBSrc
1	fedca2b7	ExtOp=1, RegWr=1, ALUBSrc=2, ALUctr=f	PC=00000400、BusW=fedca000、 Din=00000000、NPCASrc=0、 NPCBSrc=0
2	f9c28293	RegWr=1, ALUBSrc=2	PC=00000404、BusW=fedc9f9c、 Din=00000000、NPCASrc=0、 NPCBSrc=0
3	01006013	RegWr=1, ALUBSrc=2, ALUctr=6	PC=00000408、BusW=00000010、 Din=00000000、NPCASrc=0、 NPCBSrc=0
4	06502223	ExtOp=2, MemWr=1, ALUBSrc=2	PC=0000040c、BusW=00000064、 Din=fedc9f9c、NPCASrc=0、 NPCBSrc=0
5	06400303	MemOp=5, RegWr=1, ALUBSrc=2, MemtoReg=1	PC=00000410、BusW=ffffff9c、 Din=00000000、NPCASrc=0、

序号	指令	控制信号赋值，未列出的控制信号值为 0。	输出信号值 PC、BusW、Din、NPCASrc、NPCBSrc
			NPCBSrc=0
6	06601423	ExtOp=2, MemWr=1, MemOp=6, ALUBSrc=2	PC=00000414、BusW=00000068、 Din=ffffff9c、NPCASrc=0、 NPCBSrc=0
7	06405383	MemOp=2, RegWr=1, ALUBSrc=2, MemtoReg=1	PC=00000418、BusW=00009f9c、 Din=00000000、NPCASrc=0、 NPCBSrc=0
8	06701623	ExtOp=2, MemWr=1, MemOp=6, ALUBSrc=2	PC=0000041c、BusW=0000006c、 Din=00009f9c、NPCASrc=0、 NPCBSrc=0
9	4042d413	RegWr=1, ALUBSrc=2, ALUctr=d	PC=00000420、BusW=ffedc9f9、 Din=00000000、NPCASrc=0、 NPCBSrc=0
10	006444b3	RegWr=1, ALUctr=4	PC=00000424、BusW=00123665、 Din=ffffff9c、NPCASrc=0、 NPCBSrc=0
11	00649533	RegWr=1, ALUctr=1	PC=00000428、BusW=50000000、

序号	指令	控制信号赋值，未列出的控制信号值为 0。	输出信号值 PC、BusW、Din、NPCASrc、NPCBSrc
			Din=ffffff9c、NPCASrc=0、NPCBSrc=0
12	008505b3	RegWr=1	PC=0000042c、BusW=4fedc9f9、Din=ffedc9f9、NPCASrc=0、NPCBSrc=0
13	00b2a633	RegWr=1, ALUctr=2	PC=00000430、BusW=00000001、Din=4fedc9f9、NPCASrc=0、NPCBSrc=0
14	00b2b6b3	RegWr=1, ALUctr=3	PC=00000434、BusW=00000000、Din=4fedc9f9、NPCASrc=0、NPCBSrc=0
15	40b287b3	RegWr=1, ALUctr=8	PC=00000438、BusW=aeeed5a3、Din=4fedc9f9、NPCASrc=0、NPCBSrc=0
16	06f02823	ExtOp=2, MemWr=1, ALUBSrc=2	PC=0000043c、BusW=00000070、Din=aeeed5a3、NPCASrc=0、NPCBSrc=0

序号	指令	控制信号赋值，未列出的控制信号值为 0。	输出信号值 PC、BusW、Din、NPCASrc、NPCBSrc
17	0067c263	ExtOp=3, ALUctr=2, BandJ=6	PC=00000440、BusW=00000001、 Din=ffffff9c、NPCASrc=0、 NPCBSrc=1
18	0067d263	ExtOp=3, ALUctr=2, BandJ=7	PC=00000444、BusW=00000001、 Din=ffffff9c、NPCASrc=0、 NPCBSrc=0
19	0040086f	ExtOp=4, RegWr=1, ALUABSrc=1, ALUBSrc=1, BandJ=1	PC=00000448、BusW=0000044c、 Din=00000000、NPCASrc=0、 NPCBSrc=1
20	004808e7	RegWr=1, ALUABSrc=1, ALUBSrc=1, BandJ=2	PC=0000044c、BusW=00000450、 Din=00000000、NPCASrc=1、 NPCBSrc=1
21	00001917	ExtOp=1, RegWr=1, ALUABSrc=1, ALUBSrc=2	PC=00000450、BusW=00001450、 Din=00000000、NPCASrc=0、 NPCBSrc=0

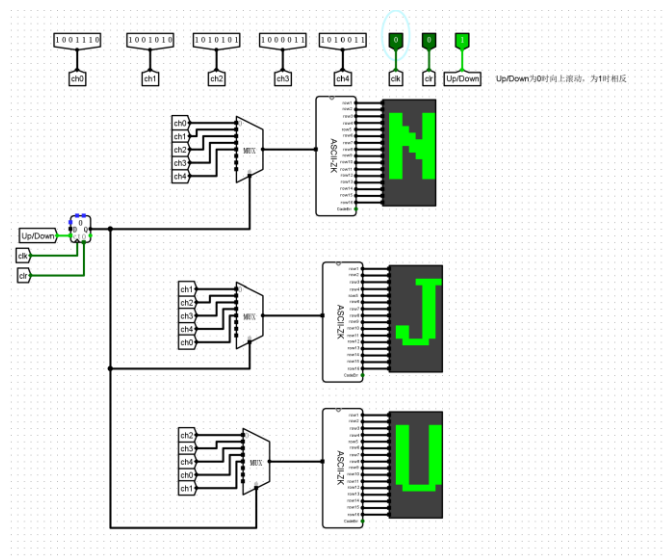
5. 错误现象及分析：

一开始没有向指令寄存器中加载 lab5.5.hex, 导致没有有效的输出。经过排查发

现 IR 为零进而发现该问题后，通过加载 hex 文件解决问题。

思考题

1. 利用三个 lab5.1 的电路，加上计数器与多路选择器，实现字符滚动。



2. PC 寄存器、寄存器堆和数据存储器写入数据的时钟信号触发边沿不一致，会导致以下影响：

- 1) **写后读数据冒险**：如果 RegFile 的写入边沿晚于 PC 的更新边沿，则相邻有数据依赖的指令会读到寄存器的旧值，导致运算结果错误。
- 2) **访存数据不一致**：如果 DataRAM 的写入边沿与 RegFile 不同步，则 store 指令可能存入错误数据，load 指令后的依赖指令可能读到过期值。
- 3) **时序违规**：如果 PC 更新边沿早于其他部件，组合逻辑路径的延迟余量减半，高频下可能无法在半个周期内完成取指和运算

3. 在硬件上加入 HALT 检测电路，在程序的末尾加入 HALT 指令，实现 CPU 的停机。（大致思路为加入 HALT 指令：0x00000000，当检测到 HALT 指令时，

IFU 输出 $\text{DataA} = \text{DataB}$, 实现停机)